Longest Repeating Character Replacement

Two approaches, a frame-by-frame walkthrough, and the core logic to remember

Brute Force · O(n²) **Sliding Window · O(n)**

Reference notes · LongestSubStringAfterReplacement.java

Problem Statement

Given a string `s` of uppercase letters and an integer `k`, you may change **any character to any other letter** at most `k` times. Return the length of the longest substring of one repeated letter you can get.

s = "AABABBA"
k = 1

→

longest window fixable
with ≤ 1 replacement

→

Output: 4
("BABB" → "BBBB")

replacements needed = window length − count of the most frequent letter

keep the most frequent letter, replace everything else ⇒ window is fixable ⇔ length − maxCount ≤ k

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	O(n²)	O(1)	Every start, extend right, accept if `length − maxCount ≤ k`
Sliding Window	O(n)	O(1)	Keep the window fixable; drop the left letter when it isn't

Big picture: same shape as the earlier window problems, but the validity test is a *formula* instead of a set/map size. Space is O(1) because the frequency array has a fixed 26 slots.

How to read the walkthrough: a PDF can't play animation, so the sliding window is shown as a **storyboard** — each frame is one tick of the loop. Read left → right, top → bottom, like a flipbook.

Frame legend

not visited · inside window · just added · needs > k replacements · dropped (outside) · best window

L = low pointer (window start) **H** = high pointer (newest char) ★ = new best length

1 · Brute Force

O(n²) time **O(1) space**

```java
// every start i, extend j, accept the window if length - maxCount <= k
private static int bruteforce(String s, int k) {
    int n = s.length();
    int res = 0;

    for (int i = 0; i < n; i++) {
        int[] freq = new int[26];
        int maxCount = 0;

        for (int j = i; j < n; j++) {
            char ch = s.charAt(j);
            freq[ch - 'A']++;

            maxCount = Math.max(maxCount, freq[ch - 'A']);

            int length = j - i + 1;
            int replacements = length - maxCount;

            if (replacements <= k) {
                res = Math.max(res, length);
            }
        }
    }
    return res;
}
```

Recall trick: "Every start, extend right, track the most frequent letter, accept the window if `length − maxCount ≤ k`." Correct, but restarts the frequency array for every `i`.

2 · Sliding Window — the code

O(n) time **O(1) space**

```java
// keep the window fixable: length - maxCount <= k; otherwise drop the left letter
private static int usingSlidingWindow(String s, int k) {
    int[] freq = new int[26];
    int low = 0, maxCount = 0, res = 0;

    for (int high = 0; high < s.length(); high++) {
        Character ch = s.charAt(high);
        freq[ch - 'A']++;
        maxCount = Math.max(maxCount, freq[ch - 'A']);
        int length = high - low + 1;
        int diff = length - maxCount;

        while (diff > k) {
            Character lowch = s.charAt(low);
            freq[lowch - 'A']--;
            low++;
            length = high - low + 1;
            diff = length - maxCount;
        }

        res = Math.max(res, length);
    }
    return res;
}
```

Why `maxCount` is never lowered: after shrinking it may be *stale* (larger than the true max in the window). That's safe — the window **never gets smaller** (each step grows or slides it), and the answer can only grow when `maxCount` *genuinely* reaches a new record. So the inner `while` runs at most once per step.

Sliding Window — frame-by-frame walkthrough

s = "AABABBA", k = 1 · read like a flipbook: left → right, top → bottom

1 Start — START



freq ∅
maxCount **0** | len **0** | best **0**
k = 1: we may change at most one letter. **L** marks the window start.

2 'A' arrives — EXPAND



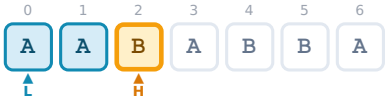
freq `A:1`
maxCount **1** | len **1** | len−max = **0 ≤ k** ✓ | best **1** ★
Window "A": 0 replacements needed (len 1 − max 1).

3 'A' arrives — EXPAND



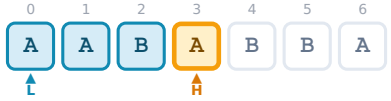
freq `A:2`
maxCount **2** | len **2** | len−max = **0 ≤ k** ✓ | best **2** ★
Window "AA": still 0 replacements.

4 'B' arrives — EXPAND



freq `A:2` `B:1`
maxCount **2** | len **3** | len−max = **1 ≤ k** ✓ | best **3** ★
Window "AAB": change the B → 1 replacement ≤ k. Fixable!

5 'A' arrives — EXPAND



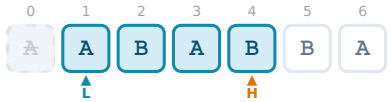
freq `A:3` `B:1`
maxCount **3** | len **4** | len−max = **1 ≤ k** ✓ | best **4** ★
Window "AABA": 3 A's, change the B → 1 replacement. Best so far = 4.

6 'B' arrives — INVALID



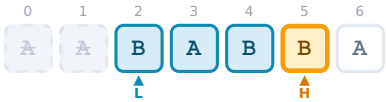
freq `A:3` `B:2`
maxCount **3** | len **5** | len−max = **2 > k** ✗ | best **4**
Window "AABAB": 5 − 3 = 2 replacements > k. **Too many** — must shrink.

7 Shrink: drop s[0] — SHRINK



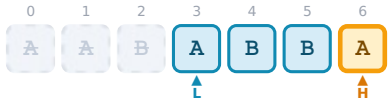
freq `A:2` `B:2`
maxCount **3** | len **4** | len−max = **1 ≤ k** ✓ | best **4**
Drop 'A', low → 1. Window slides at size 4. **maxCount stays 3** (never lowered).

8 'B' arrives → shrink — SHRINK



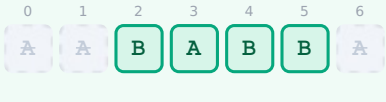
freq `A:1` `B:3`
maxCount **3** | len **4** | len−max = **1 ≤ k** ✓ | best **4**
Len 5, diff 2 > k → drop 'A', low → 2. Window "BABB" is truly valid. Best stays 4.

9 'A' arrives → shrink — SHRINK



freq `A:2` `B:2`
maxCount **3** | len **4** | len−max = **1 ≤ k** ✓ | best **4**
Len 5, diff 2 > k → drop 'B', low → 3. End of string — answer = **4**.

✓ Result — RESULT



longest fixable window = **4**
e.g. "BABB" → replace 1 letter → "BBBB"
Longest window = **4**. The window never shrinks below the best size found.

TL;DR — For Future Me

- **Best solution:** sliding window, O(n) — keep the window fixable: `length − maxCount ≤ k`; otherwise drop the left letter.
- **Core formula:** replacements needed = window length − count of the most frequent letter.
- **Trick:** never lower `maxCount` — the window never shrinks, and the answer only grows when `maxCount` truly grows.
- **Pattern:** "longest window needing at most k fixes" — same family as *Max Consecutive Ones III* (flip at most k zeros).